

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Пишем фреймворк

Фреймворк (**framework**) переводится как "каркас". То есть это каркас, фундамент, на котором базируются какие-то классы, модули, пакеты, приложения. Фреймворк облегчает процесс создания своих каких-то приложений, например создание сайта. В языке Python для этой цели наиболее известны фреймворки Django и Flask.

В самом простом виде фреймворк это набор, состоящий из ряда программ, относящихся к какой-то одной области. То есть при запуске фреймворка вы увидите меню, в котором предлагается запуск той или иной программы. Каждая программа написана отдельно, и с помощью импорта подключается из модуля или пакета.

Проще говоря, фреймворк может представлять из себя платформу, конструктор, помогающий реализовать что-то своё. Или может содержать готовые компоненты, программы, объединённый в эдакий комбайн, чтобы выполнять целый ряд задач, что гораздо удобнее, чем множество программ по-отдельности.

Напишем простой мини-фреймворк на тему **OSINT** - **Open-Source INTelligence** (разведка на основе открытых источников).

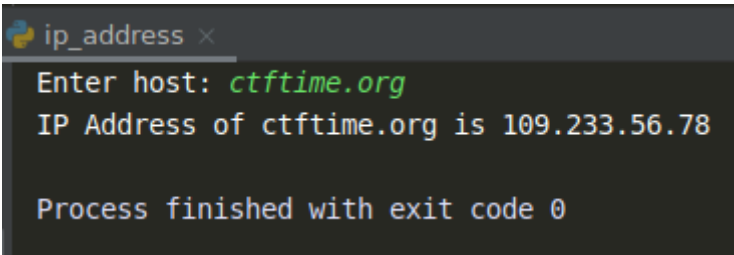
В первую очередь необходимо составить план, что будет входить в наш фреймворк:

1. Определение IP сайта
2. Поиск по IP таких данных как страна, координаты, часовой пояс, название континента, города, кода страны и т.п.
3. Данные whois домена
4. Данные nslookup домена
5. DNS MX-Record
6. Reverse DNS
7. Содержимое файла robots.txt при его наличии
8. Содержимое файла sitemap.xml при его наличии

Список можно продолжать, но этого будет вполне достаточно, чтобы сделать образец, и разобраться в структуре.

Сначала мы напишем 8 отдельных скриптов:

1. Host IP



```
ip_address x
Enter host: ctftime.org
IP Address of ctftime.org is 109.233.56.78

Process finished with exit code 0
```

Чтобы программа работала корректно, предварительно очищаем имя хоста от протокола и последнего слеша, если пользователь введёт адрес ресурса с ними.

```
import socket

host = input('Enter host: ') # ctftime.org
if '://' in host:
    host = host.split('://')[1]
host = host.replace('/', '')
remote_ip = socket.gethostbyname(host)
print(f"{'IP Address of '}{'{host}'}{' is '}{'{remote_ip}'}")
```

ip_address x

Enter host: https://ctftime.org/
IP Address of ctftime.org is 109.233.56.78

Process finished with exit code 0

2. Site location

По IP который мы определили в предыдущей программе, получаем ряд сведений.

```
geo x
Enter ip: 109.233.56.78
country_code: NL
country_code3: NLD
country_name: Netherlands
continent: EU
region_code: 11
city: Group
latitude: 51.7833
longitude: 4.433300000000003
time_zone: Europe/Amsterdam

Process finished with exit code 0
```

Данные берутся из локальной базы [GeoIPCity.dat](#). Так как сведения для разных IP отличаются полнотой информации, то поставим условие показать только ту часть данных, которая присутствует.

Чтобы посмотреть все параметры, имеющиеся в базе, нужно раскомментировать строку `print(city)`.

```
import pygeoip

ip = input('Enter ip: ') # 109.233.56.78
gi = pygeoip.GeoIP('/root/PycharmProjects/education/GeoIPCity.dat')
city = gi.record_by_addr(ip)
# print(city)
for key in city:
    if city[key] is None or city[key] == 0:
        continue
    else:
        print(key, city[key], sep=": ")
```

3. Whois

Сервис Whois позволяет быстро получить всю информацию о регистрации домена, например, дату регистрации и возраст домена, или узнать контакты, по которым можно связаться с организацией или человеком. Часть сведений могут быть приватными, и поэтому недоступными. Модуль [whois](#) поддерживает большинство доменных имён.

```
import whois

add = input('Enter domain: ') # ctftime.org
domain = whois.query(add)
res = domain.__dict__
for i in res:
    print(i, res[i], sep=': ')

whois_add x
Enter domain: ctftime.org
name: ctftime.org
registrar: NameCheap, Inc.
creation_date: 2012-04-19 16:23:07
expiration_date: 2021-04-19 16:23:07
last_updated: None
status: ok https://icann.org/epp#ok
name_servers: {'dns1.yandex.net', 'dns2.yandex.net'}

Process finished with exit code 0
```

4. Nslookup

Эта программа для запроса к серверам **DNS** (доменная система имён), предоставляет информацию о различных DNS записях домена.


```

site_nslookup x
Enter host: ctftime.org
ctftime.org. 21600 IN A 109.233.56.78
ctftime.org
900 IN SOA dns1.yandex.net
dns-hosting.yandex.ru
9 900 90 86400 900

Process finished with exit code 0

```

Так же выводится DNS SOA запись — [Start of Authority](#). Для каждого домена SOA-запись создается автоматически.

```

from nslookup import Nslookup

domain = input('Enter host: ') # ctftime.org

# set optional Cloudflare public DNS server
dns_query = Nslookup(dns_servers=["1.1.1.1"])

ips_record = dns_query.dns_lookup(domain)
for i in ips_record.response_full:
    print(i)

soa_record = dns_query.soa_lookup(domain)
for i in soa_record.response_full:
    print('\n'.join(i.split('. ')))

```

5. DNS MX-Record

MX-запись указывает на сервер, принимающий почту для домена.

```

import dns.resolver

address = input('Enter host: ') # ctftime.org
my_resolver = dns.resolver.Resolver(configure=False)
my_resolver.nameservers = ['8.8.8.8', '1.1.1.1']
answers = my_resolver.query(address, 'MX')

for rdata in answers:
    print('MX Record:', rdata.exchange)

# pip install dnspython3

```

```

dns_resolver x
Enter host: ctftime.org
MX Record: mx.yandex.ru.

```

6. Reverse DNS

Обратный просмотр DNS — обращение к особой доменной зоне **для определения имени узла** (устройство, соединённое с другими устройствами как часть сети) по его IP-адресу с помощью PTR-записи.

Фактически **PTR** означает POINTER, и является уникальной записью в системе DNS, которая связывает IP-адрес с каноническим именем хоста. Практически PTR-запись является частью системы DNS, но создается с помощью специального файла зоны с классическим форматом **in-addr.arpa**.

На одном и том же IP-адресе может быть множество хостов. И если в файле конфигурации не указана обратная зона, то определить имя узла таким образом не получится.

```
from dns import reversename, resolver

ip = input('Enter ip: ') # 77.88.55.55
rev_name = reversename.from_address(ip)
try:
    reversed_dns = str(resolver.query(rev_name, "PTR")[0])
    print(reversed_dns)
except Exception as e:
    print(e)
```

reversed_dns x

Enter ip: 109.233.56.78

The DNS query name does not exist: 78.56.233.109.in-addr.arpa.

И если PTR-запись есть, то мы получим имя домена.

```
reversed_dns x
```

Enter ip: 77.88.55.55

yandex.ru.

Process finished with exit code 0

7. robots.txt

Robots.txt — это текстовый файл, который содержит параметры индексирования сайта для роботов поисковых систем. Зачастую в этом файле можно обнаружить директории и файлы запрещённые к индексированию, и среди них скрытые, что может потенциально упростить взлом.

```

robots x
Enter host [https://site.com]: https://ctftime.org
User-agent: *
Crawl-delay: 10

Disallow: /user/
Disallow: /task/tags/
Disallow: /login/

# public data for rating experiments
Disallow: /export/

# secret page
Disallow: /dating/

User-agent: dotbot
Disallow: /

User-agent: SemrushBot
Disallow: /

User-agent: SemrushBot-SA
Disallow: /

User-agent: MJ12bot

```

```

import requests

def get_page_data(html):
    res = html.text
    print(res)

if __name__ == '__main__':
    url = input('Enter host [https://site.com]: ') # https://ctftime.org
    head = {"User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 "
                        "(KHTML, like Gecko) Chrome/65.0.3325.181 Safari/537.36"}
    if url[-1] == '/':
        page = requests.get(url + 'robots.txt', headers=head)
    else:
        page = requests.get(url + '/robots.txt', headers=head)
    if page.status_code != 404:
        get_page_data(page)
    else:
        print('File "robots.txt" not found!')

```

8. sitemap.xml

Sitemap — это файл со ссылками на страницы сайта, который сообщает поисковым системам об актуальной структуре сайта. Поддерживает форматы XML и TXT. В реальности формат txt довольно редко используется на практике для составления карты сайта. В код не вошла проверка sitemap.txt, но при желании её можно легко добавить.

Для пентестера наличие карты сайта облегчает понимание объёма сайта, а также представляет отличный ссылочный материал для проверки на уязвимости, без необходимости парсить сайт.

```
sitemap x
Enter host [https://site.com]: https://ctftime.org
File "sitemap.xml" not found!

Process finished with exit code 0
```

```
sitemap x
Enter host [https://site.com]: https://edu.hackeru.pro/
https://edu.hackeru.pro/
https://edu.hackeru.pro/windowsforensics
https://edu.hackeru.pro/windowsforensics_b
https://edu.hackeru.pro/ceh
https://edu.hackeru.pro/ceh_b
https://edu.hackeru.pro/alfabank
https://edu.hackeru.pro/alfabank_b
https://edu.hackeru.pro/internetmarketing
https://edu.hackeru.pro/internetmarketing_b
https://edu.hackeru.pro/webdevelopment
https://edu.hackeru.pro/webdevelopment_b
https://edu.hackeru.pro/cybersecurity
https://edu.hackeru.pro/cybersecurity_b
https://edu.hackeru.pro/cyber_hse
https://edu.hackeru.pro/cyber_hse_b
https://edu.hackeru.pro/mobile
https://edu.hackeru.pro/ceh_thank_you
https://edu.hackeru.pro/spetsialist_po_internet_marketingu_thank_you
https://edu.hackeru.pro/spetsialist_po_razrabotke_frontend_backend_thank_you
https://edu.hackeru.pro/spetsialist_po_web_razrabotke_thank_you
https://edu.hackeru.pro/kompyuternaya_kriminalistika_v_windows_thank_you
https://edu.hackeru.pro/spetsialist_po_informatsionnoy_bezopasnosti_thank_you
https://edu.hackeru.pro/eduhackerupro thank you
```



```
import requests
from bs4 import BeautifulSoup

def get_page_data(html):
    res = BeautifulSoup(html.text, 'lxml')
    line = res.find_all("loc")
    for i in line:
        print(i.text)

if __name__ == '__main__':
    url = input('Enter host [https://site.com]: ')
    if url[-1] == '/':
        page = requests.get(url + 'sitemap.xml')
    else:
        page = requests.get(url + '/sitemap.xml')
    if page.status_code == 200:
        get_page_data(page)
    else:
        print('File "sitemap.xml" not found!')
```

Ну что же, скрипты написаны, настало время их объединить. Создадим папку **full_scripts**, в которую мы переместим все наши скрипты. Также создадим файл **frame.py** – это будет наш главный файл, который будет лежать рядом с папкой **full_scripts**, то есть в одной директории с ним.

В главном файле сделаем вывод меню и логотипа.

[illegible]

Выбор опций осуществляется в бесконечном цикле, для выхода из программы выбирается ноль.

```
frame x
5. DNS MX-record
6. Reverse DNS
7. robots.txt
8. sitemap.xml

Enter the option number: 0
The program is complete!

Process finished with exit code 0
```

Чтобы программы смогли работать из главного файла frame.py, нужно будет их подключить как модули через импорт.

Делать это будем по очереди, по одному скрипту, так как нам придётся все скрипты переписывать с использованием функций.

```
import socket

def address():
    host = input('Enter host: ') # ctftime.org
    if '://' in host:
        host = host.split('://')[1]
    host = host.replace('/', '')
    remote_ip = socket.gethostbyname(host)
    print(f'IP Address of '{host}' is '{remote_ip}')
```

Поскольку импорт осуществляется из папки, то он делается через точку с названием папки. Соответственно для файла `ip_address.py` импорт будет выглядеть так: `import full_scripts.ip_address`

В самом коде, чтобы использовать нужную функцию, требуется её дописать через точку.

```
while True:
    num = int(input('Enter the option number: '))
    if num == 0:
        print('The program is complete!')
        exit(0)
    elif num == 1:
        full_scripts.ip_address.address()
```

Не забывайте, что для удобства можно использовать короткие псевдонимы импортируемых модулей. Например, `import full_scripts.ip_address as add` тогда далее в коде будет `add.address()`

Шапка программы может выглядеть так:

```
import full_scripts.ip_address as add


logo = "\033[32m" + r'''
_ _ _ _ _ 
/_ \__|_| \| | |_|_) /_\ \/_/
|| |\__ \| || \| | | | |_) \| | \| /
|| | |_) | || \| | | | |_) | | | / \
\_\/ |_/ | | \| | | | |_) / \_/\ \

...

print(logo)
menu = '''
0. Exit the program
1. Host IP
2. Site location
3. Whois
4. Nslookup
5. DNS MX-Record
6. Reverse DNS
7. robots.txt
8. sitemap.xml
''' + "\033[0m"

print(menu)

while True:
```

Все остальные программы подключаются точно также. И чтобы данные не сливались, сделаем какие-нибудь разделители + пустые строки, например так:

```
while True:
    num = int(input('\nEnter the option number: '))
    if num == 0:
        print('\nThe program is complete!')
        exit(0)
    elif num == 1:
        print('-'*35, '1. Host IP', '-'*35, sep='\n')
        add.address()
    elif num == 2:
        print('-'*35, '2. Site location', '-'*35, sep='\n')
        geo.geo_ip()
```

```
-----  
1. Host IP  
-----  
Enter host: 109.233.56.78  
IP Address of 109.233.56.78 is 109.233.56.78  
  
Enter the option number: 2  
-----  
2. Site location  
-----  
Enter ip: 109.233.56.78  
country_code: NL  
country_code3: NLD  
country_name: Netherlands  
continent: EU  
region_code: 11  
city: Group  
latitude: 51.7833  
longitude: 4.433300000000003  
time_zone: Europe/Amsterdam  
  
Enter the option number: 0  
  
The program is complete!
```

И не забываем, что в код нужно добавить обработку ошибок при выборе некорректной опции.